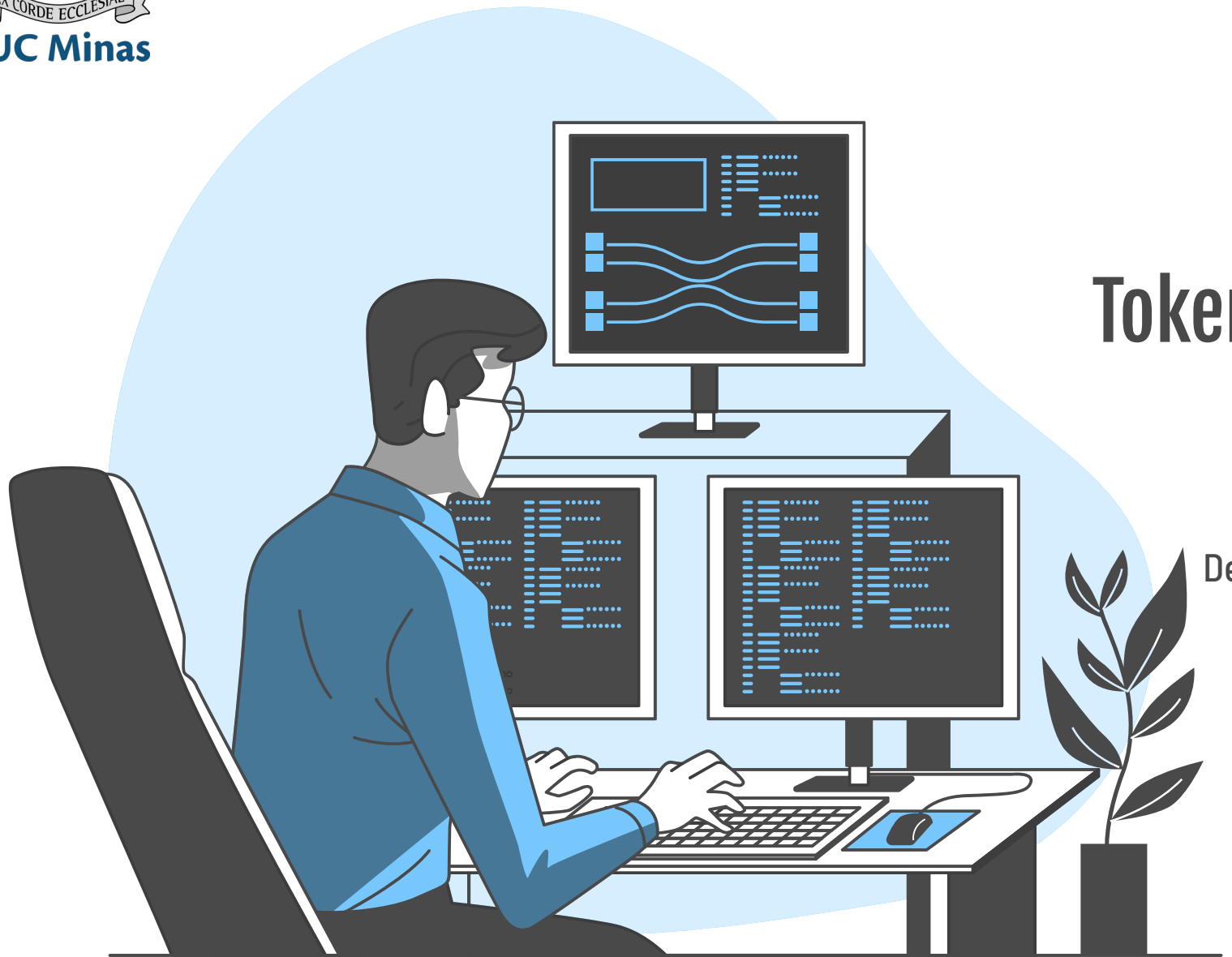
PUC Minas



Aula 10 · Back-end

Token JWT: autenticação sem sessão

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender as três partes de um **JWT** e o que vai no *payload*

2

Gerar e validar tokens com a biblioteca **JJWT**

3

Criar o endpoint `POST /login` que devolve o token

4

Escrever o filtro `JwtAuthFilter` e registrá-lo na `SecurityConfig`

5

Testar rotas protegidas com `Authorization: Bearer`

Apresentação da aula

Antes de começar

- Aula 09: Spring Security

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Por que token?

02 Anatomia de um JWT

03 Dependências

04 A chave secreta

05 JwtUtil: gerar e validar

06 Endpoint de login

07 O filtro que valida o token

08 Registrando o filtro na SecurityConfig

09 Testando

10 Aprofundando: JWT em produção

01

Por que token?

Por que token?

Com sessão, o servidor guarda quem está logado e o navegador manda um cookie. Isso complica quando o front está em outro domínio (Vercel) ou quando há vários servidores. Com **JWT** (*JSON Web Token*), o servidor **não guarda nada**: no login ele entrega um token assinado, e o cliente envia esse token em toda requisição. Se a assinatura confere, o servidor confia no conteúdo. Isso se chama autenticação **stateless**.

01

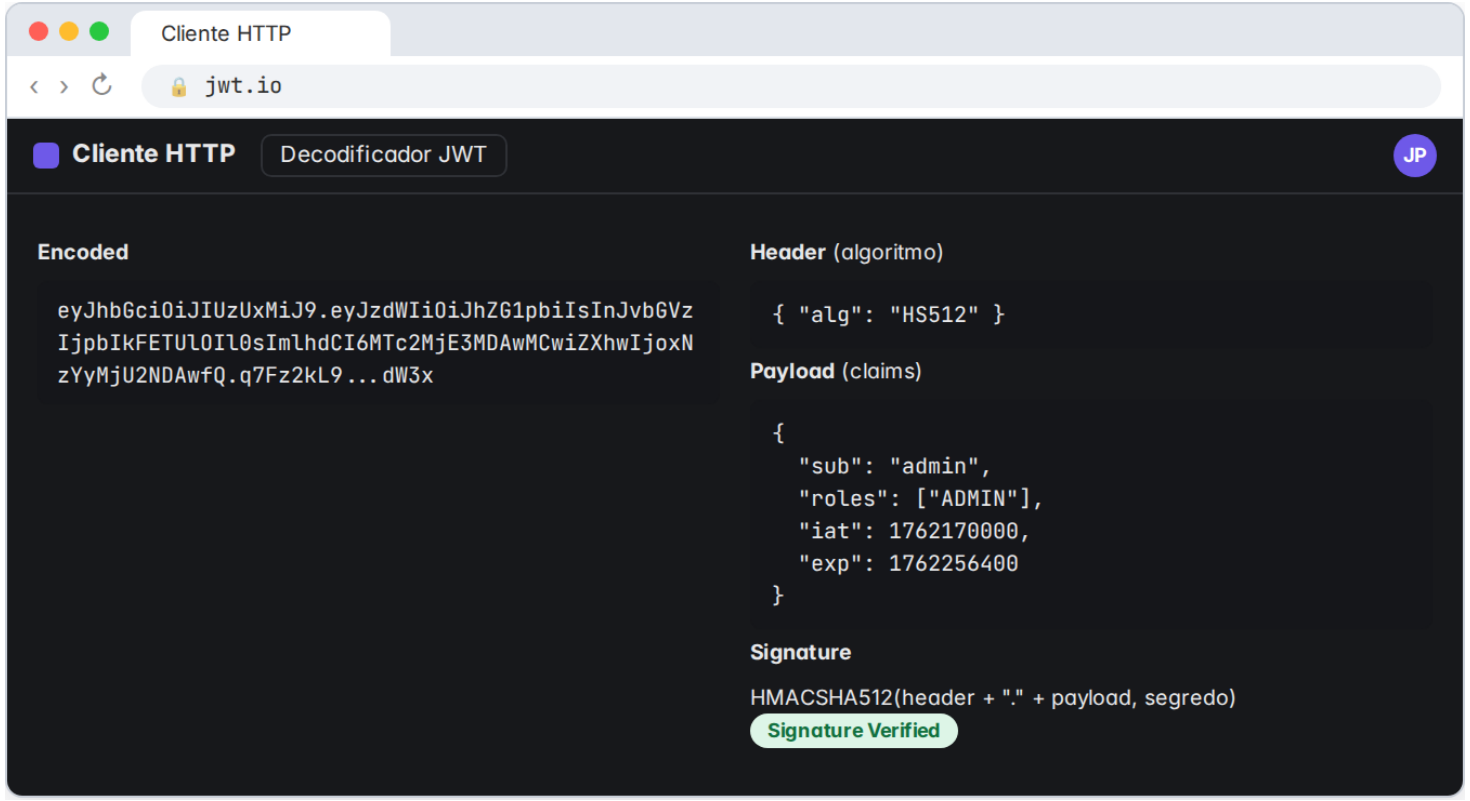
Por que token?

Aula 10 · Back-end

02

Anatomia de um JWT

Anatomia de um JWT



Um JWT são três textos em Base64URL separados por pontos: `header.payload.signature`.

Decodificando um token: o *payload* é só Base64, qualquer um consegue ler.

Anatomia de um JWT

Claim	Significado
`sub`	*Subject*: quem é o usuário (username)
`iat`	*Issued at*: quando foi gerado
`exp`	*Expiration*: quando expira
`roles`	Claim própria com os papéis

Anatomia de um JWT

Atenção

O *payload* **não é criptografado**, só codificado. Nunca coloque senha ou dados sensíveis no token. A assinatura garante que ninguém **alterou** o conteúdo, não que ninguém **leu**.

03

Dependências

Dependências

pom.xml

```xml
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-gson</artifactId>   <!-- gson evita conflito com o Jackson 3 do Boot 4 -->
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
```

04

A chave secreta

A chave secreta

O algoritmo **HS512** precisa de uma chave de pelo menos 512 bits (64 bytes). Os projetos antigos (`JwtUtil` do `JWT_RestAPI`) geram uma chave nova a cada execução, o que **invalida todos os tokens quando o servidor reinicia**, algo que acontece o tempo todo no Render gratuito. O correto é ler a chave de uma variável de ambiente:

04

A chave secreta

Aula 10 · Back-end

A chave secreta

Gerando um segredo

```
# gere uma vez e guarde (Linux/macOS/Git Bash)
openssl rand -base64 64
```

A chave secreta

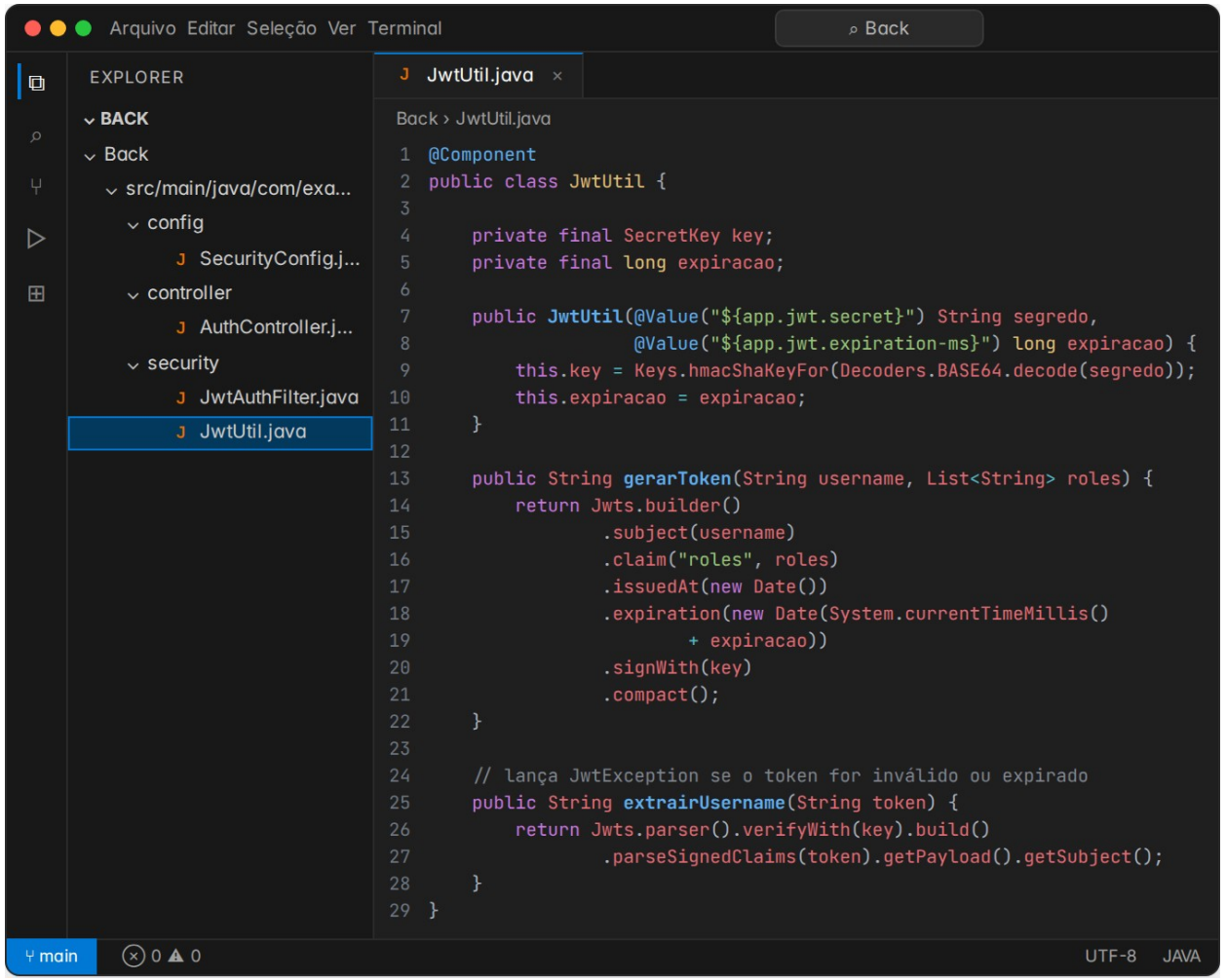
application.properties

```
app.jwt.secret=${JWT_SECRET:ZGV2LXNvbWVudGUtdHJvcXVlLWVzdGEtY2hhdmUtcG9yLXVtYS1nZXJhZGEtY29tLW9wZW5zc2wtcmFuZC1iYXNlNjQtNjQ=}
app.jwt.expiration-ms=86400000
```

05

JwtUtil: gerar e validar

JwtUtil: gerar e validar



`JwtUtil` no pacote `security`: a chave vem do `application.properties`.

06

Endpoint de login

Endpoint de login

controller/AuthController.java

```java
@RestController
public class AuthController {
    private final AuthenticationManager authManager;
    private final JwtUtil jwt;
    // construtor...

    @PostMapping("/login")
    public LoginResponseDTO login(@RequestBody LoginRequestDTO req) {
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.username(), req.password()));
        List<String> roles = auth.getAuthorities().stream()
            .map(a -> a.getAuthority().replace("ROLE_", "")).toList();
        return new LoginResponseDTO(jwt.gerarToken(auth.getName(), roles), roles);
    }
}
```

Endpoint de login

Se usuário ou senha estiverem errados, o `authenticate` lança `BadCredentialsException`, que o Spring converte em **401**.

06

Endpoint de login

Aula 10 · Back-end

07

O filtro que valida o token

O filtro que valida o token

Em cada requisição, o filtro lê o cabeçalho `Authorization`, valida o token e, se estiver tudo certo, diz ao Spring quem é o usuário:

security/JwtAuthFilter.java

```java
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtUtil jwt;
    private final UserDetailsService uds;
    // construtor...

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
```

O filtro que valida o token (cont.)

security/JwtAuthFilter.java

```
                String username = jwt.extrairUsername(header.substring(7));
                UserDetails user = uds.loadUserByUsername(username);
                var auth = new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                // token inválido ou expirado: segue sem autenticar → 401
            }
        }
        chain.doFilter(req, res);
    }
}
```

08

Registrando o filtro na SecurityConfig

Registrando o filtro na SecurityConfig

config/SecurityConfig.java

```
http
    .csrf(csrf -> csrf.disable())
    .cors(Customizer.withDefaults())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.POST, "/login").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated())
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

Registrando o filtro na SecurityConfig

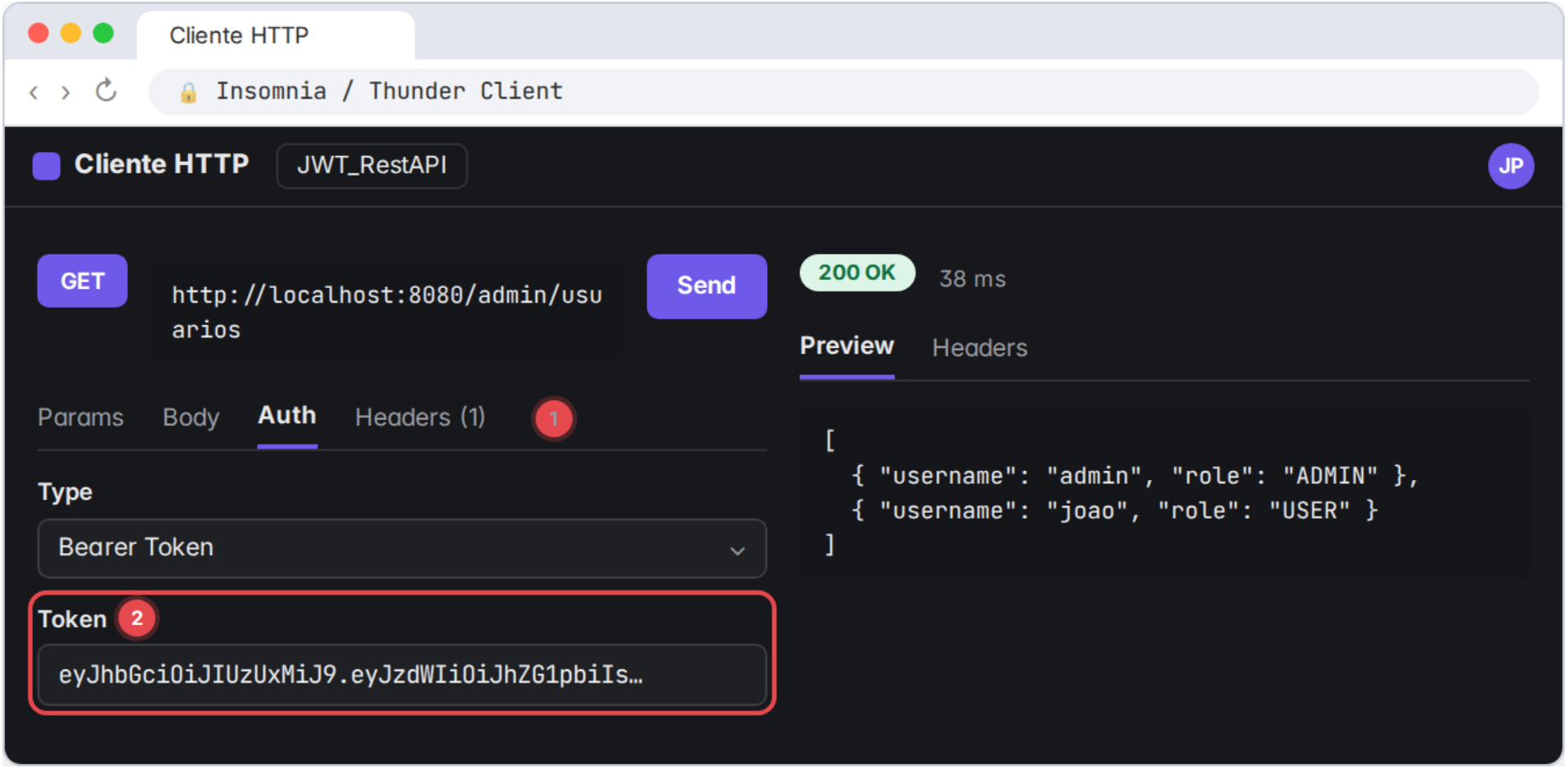


Fluxo completo de autenticação com JWT.

09

Testando

Testando



Rota protegida acessada com o token na aba **Auth → Bearer Token**.

Testando

Com curl

```
# 1) login
curl -s -X POST localhost:8080/login -H "Content-Type: application/json" \
     -d '{"username":"admin","password":"1234"}'
# {"token":"eyJhbGciOiJIUzUxMiJ9...","roles":["ADMIN"]}

# 2) usar o token
curl localhost:8080/admin/usuarios -H "Authorization: Bearer eyJhbGciOiJIUzUxMiJ9..."
```

Testando

Erro comum

401 mesmo com o token: falta o prefixo `Bearer ` (com espaço), o token expirou, ou o servidor reiniciou com outra chave (chave gerada em memória). **403 com token válido**: o usuário não tem o papel exigido; lembre que `hasRole("ADMIN")` procura `ROLE_ADMIN` nas *authorities*.

10

Aprofundando: JWT em produção

HS256/HS512 x RS256

Simétrico (HS256, HS512)

- Uma chave secreta assina **e** valida
- Simples: uma variável `JWT_SECRET`
- Ideal quando só a sua API valida o token

vs

Assimétrico (RS256, ES256)

- Chave privada assina, chave pública valida
- Outros serviços validam sem poder emitir
- Usado por provedores (Google, Auth0, Supabase Auth)

Access token e refresh token

Tokens de longa duração são perigosos se vazarem. O padrão profissional usa dois: um **access token** curto (5 a 15 minutos) enviado em toda requisição, e um **refresh token** longo (dias), guardado com mais cuidado, usado só para pedir um novo access token em `POST /api/refresh`.

10

Aprofundando: JWT em produção

Aula 10 · Back-end

Access token e refresh token

1. Login devolve `accessToken` (15 min) e `refreshToken` (7 dias).
2. O front usa o `accessToken` no cabeçalho `Authorization`.
3. Quando a API responde 401 por expiração, o front chama `/api/refresh` com o `refreshToken`.
4. A API valida o refresh token (e se ele não foi revogado) e devolve um novo `accessToken`.
5. No logout, o refresh token é apagado/revogado no banco.

Ataques e como se proteger

Segredo fraco

Chave curta pode ser descoberta por força bruta. Use 64 bytes aleatórios (`openssl rand -base64 64`).

Algoritmo "none"

Bibliotecas antigas aceitavam token sem assinatura. O JJWT recusa por padrão.

Token vazado

Expiração curta limita o estrago. Nunca logue o token completo.

XSS

Script injetado lê o `localStorage`. Escape o que vem do usuário; evite `dangerouslySetInnerHTML`.

Lendo o usuário logado no controller

controller/AuthController.java

```java
@GetMapping("/api/me")
public Map<String, Object> me(@AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "username", user.getUsername(),
        "roles", user.getAuthorities().stream().map(GrantedAuthority::getAuthority).toList());
}

// alternativa em qualquer lugar do código:
String username = SecurityContextHolder.getContext().getAuthentication().getName();
```

Onde o cliente guarda o token

Cabeçalho Authorization (Bearer)

- Front guarda em memória ou `localStorage`
- Sem CSRF: o navegador não envia sozinho
- Vulnerável a XSS se guardado no `localStorage`

vs

Cookie HttpOnly

- O navegador envia sozinho; JavaScript não lê
- Protegido contra roubo por XSS
- Exige CSRF e configuração de cookies entre domínios

Logout com JWT

Um JWT continua válido até expirar, mesmo depois do logout, porque o servidor não guarda sessão. Opções: expiração curta (o mais simples), lista de tokens revogados no banco consultada pelo filtro, ou trocar o segredo (invalida **todos** os tokens de todos os usuários).

10

Aprofundando: JWT em produção

Aula 10 · Back-end

Logout com JWT

Revogação simples com o claim jti

```
// no JwtAuthFilter, depois de validar a assinatura:
Claims claims = jwt.claims(token);
if (tokenRevogadoRepository.existsById(claims.getId())) {   // claim "jti"
    chain.doFilter(req, res);   // segue sem autenticar → 401
    return;
}
```

Aprofundando: JWT em produção

Claim padrão	Uso
`iss` (issuer)	Quem emitiu: valide para recusar tokens de outro sistema
`aud` (audience)	Para quem o token foi emitido
`jti` (JWT id)	Identificador único: permite revogar um token específico
`nbf` (not before)	O token só vale a partir dessa data

Aprofundando: JWT em produção

Na prática

Compare com o **Supabase Auth**: ele também emite JWTs. Em projetos sem back-end próprio, o RLS do Postgres lê o `auth.uid()` de dentro desse token para decidir o que cada usuário pode ver.

11

Quiz de revisão

Pergunta

Quais são as três partes de um JWT?

A header.payload.signature

B user.password.role

C key.value.hash

D id.token.secret

Resposta

Quais são as três partes de um JWT?

✓ header.payload.signature

B user.password.role

C key.value.hash

D id.token.secret

Por quê: Separadas por pontos e codificadas em Base64URL.

Pergunta

O payload de um JWT é criptografado?

A Sim, sempre

B Não, só codificado; qualquer um consegue ler

C Só com HS512

D Só no Render

Resposta

O payload de um JWT é criptografado?

A Sim, sempre

✓ Não, só codificado; qualquer um consegue ler

C Só com HS512

D Só no Render

Por quê: Nunca coloque senha ou dado sensível no token.

Pergunta

Por que gerar a chave secreta a cada execução é um problema no Render?

A Fica lento

B Ao reiniciar, todos os tokens emitidos deixam de valer

C O Render bloqueia

D Gera CORS

Resposta

Por que gerar a chave secreta a cada execução é um problema no Render?

A. Fica lento

✓ Ao reiniciar, todos os tokens emitidos deixam de valer

C. O Render bloqueia

D. Gera CORS

Por quê: Leia a chave de `JWT_SECRET`.

Pergunta

O que o `JwtAuthFilter` faz?

A. Gera o token no login

B. Lê o Bearer, valida e preenche o SecurityContext

C. Criptografa a senha

D. Libera o CORS

Quiz · questão 4 de 5

Resposta

O que o `JwtAuthFilter` faz?

A — Gera o token no login

✓ — Lê o Bearer, valida e preenche o SecurityContext

C — Criptografa a senha

D — Libera o CORS

Por quê: Ele roda antes do controller em toda requisição.

Pergunta

Qual configuração indica autenticação sem sessão?

A `SessionCreationPolicy.STATELESS`

B `formLogin()`

C `httpBasic()`

D `csrf().enable()`

Resposta

Qual configuração indica autenticação sem sessão?

✓ `SessionCreationPolicy.STATELESS`

B `formLogin()`

C `httpBasic()`

D `csrf().enable()`

Por quê: O servidor não guarda estado de login entre requisições.

Fechamento

Resumo da aula

- ✓ JWT = `header.payload.signature`; o *payload* é legível, a assinatura impede alteração.
- ✓ A chave HS512 vem de `JWT_SECRET` (variável de ambiente), nunca gerada a cada execução.
- ✓ `POST /login` autentica com o `AuthenticationManager` e devolve o token.
- ✓ O `JwtAuthFilter` (`OncePerRequestFilter`) valida o `Bearer` e preenche o `SecurityContext`.
- ✓ Na `SecurityConfig`: CSRF desligado, sessão **STATELESS** e `addFilterBefore`.

Exercícios

1. Implemente o login JWT no CRUD de alunos e proteja `POST/PUT/DELETE`.

2. Faça o token expirar em 2 minutos e observe o 401 depois desse tempo.

3. Adicione a claim `nome` ao token e decodifique no jwt.io.

4. Crie `GET /me` que devolve o username do token usando `@AuthenticationPrincipal`.

Para ir além

- jwt.io: decodificador e introdução
- github.com/jwtk/jjwt: documentação da biblioteca
- Projetos `JWT_RestAPI`, `Documentation_JWT_RestAPI` e `Exemplo_Arquitetura_Completa`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 10?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web